Yuho: A Domain-Specific Language for Encoding the Singapore Penal Code as Executable Statute

Grammar, Tooling, and a 524-Section Proof of Concept

Gabriel Ong Zhe Mian
gabrielzmong@gmail.com
Independent, Singapore

April 25, 2026

Abstract

We present *Yuho*, a domain-specific language for encoding statutes as executable, machine-checkable artefacts, and a corresponding proof-of-concept encoding of the Singapore Penal Code 1871 (524 sections, 524 encoded, 524 passing parse-and-build, 524 human-stamped at the highest fidelity tier). The Yuho grammar models the structural primitives of criminal law: section headers and amendment lineage; elements partitioned into *actus reus*, *mens rea*, and circumstance with optional burden qualifiers; penalty combinators capturing cumulative, alternative, conditional, and nested punishment regimes; verbatim illustrations; Catala-style prioritised exceptions; and case-law anchors. A tree-sitter parser, a Python AST, six transpilers (JSON, controlled English, LaTeX, Mermaid, Alloy, DOCX), a Language Server, a Model Context Protocol server, and a Z3/Alloy verification hookup turn an encoded statute into a queryable, diffable, formally-checkable object. We discuss fourteen grammar gaps surfaced by mass-encoding the Penal Code, the linting and fidelity-checking infrastructure built to keep encodings honest, and the engineering tradeoffs involved in modelling a single common-law jurisdiction in production-strength detail rather than a synthetic toy.

1 Introduction

Statutes are not free-form prose. A modern penal code is drafted to a tight internal grammar: numbered sections; enumerated elements; in-line illustration blocks; cross-cutting general exceptions; penalty clauses with named combinators ("or both"). Lawyers learn to read this structure as if it were a typed program, and computer scientists have, since at least Sergot et al.'s 1986 encoding of the British Nationality Act [**?**], observed that the structure is amenable to direct symbolic representation. The persistent gap is not the recognition that statute looks like code — it is the practical absence of a representation expressive enough to encode an entire production statute without immediately falling back on free-text strings.

This paper argues that the gap is mostly a grammar gap, and presents *Yuho*, a domain-specific language designed by mass-encoding the entire Singapore Penal Code 1871. The headline number is engineering rather than methodological: 524 sections, 524 encoded, 524/524 passing parse-and-build (the L1+L2 tiers of our coverage harness), and 524 carrying a structured human-fidelity stamp (L3) at the time of writing. The interesting numbers sit underneath: every section that the grammar could not natively express is documented as a numbered *grammar gap* (`G1`–`G14`), and we report which were genuine grammar limitations, which were diagnostic issues misread as grammar issues, and which we resolved versus deferred.

Why a fresh DSL? Existing legal-encoding work clusters into three camps. The logic-programming tradition treats statute as Horn clauses [**?**, **?**]; the markup tradition treats statute as structured XML for archival and exchange (Akoma Ntoso [**?**], LegalRuleML [**?**]); and the modern DSL tradition picks a sub-domain — French tax code (Catala [**?**]), contracts and regulations (lam4 [**?**]), generic statute compilation (LexScript [**?**]), Word-driven authoring (LDOC [**?**]) — and builds a focused syntax for it. None of these target a common-law *penal* code at full coverage, and none expose primitives that match how penal sections are actually drafted: an offence is a typed conjunction of elements; a penalty is a small algebra over imprisonment / fine / caning / death; an exception is a prioritised override; an illustration is a verbatim worked example tied to the parent section; an amendment is a new `effective <date>` clause that does not erase the prior text.

Contributions. We make four contributions.

1. **The Yuho grammar.** A statute-shaped syntax in which the top-level construct is a `statute N { ...}` block with first-class fields for definitions, elements, penalty, illustrations, exceptions, case law, parties, and amendment lineage. Elements carry a deontic type (`actus_reus` / `mens_rea` / `circumstance` / `obligation` / `prohibition` / `permission`), an optional burden qualifier (`burden prosecution` / `burden defence`) with a named proof standard, and optional causal (`causedBy`) and temporal (`precedes` / `during` / `after`) edges

to other elements. Penalty clauses compose three top-level combinators (`cumulative` / `alternative` / `or_both`) plus a `when <ident>` conditional and a nested-combinator escape hatch. Exceptions form a Catala-style priority DAG with optional `defeats` edges to the elements they override.

2. **A complete public encoding.** All 524 sections of the Singapore Penal Code 1871, version-controlled under `library/penal_code/`, with a per-section directory layout (`statute.yh`, `metadata.toml`, `test_statute.yh`). The encoding is canonical: the `.yh` file is the source of truth; all rendered formats are generated.

3. **Tooling.** A tree-sitter parser, an immutable Python AST, six transpilers (JSON, controlled English, LaTeX, Mermaid, Alloy, DOCX), a Z3-and-Alloy verification hookup, a fidelity-diagnostic pass that cross-checks encodings against the canonical scrape, a Language Server exposing hover / inlay-hint / completion / code-lens / diagnostics, a Model Context Protocol server for AI-driven encoding workflows, and a VS Code extension wrapping the Language Server.

4. **Empirical findings.** A catalogue of fourteen distinct grammar gaps surfaced during mass-encoding, classified into ten genuine grammar limitations (all resolved by extending the parser) and four diagnostic-layer issues (all resolved by adding fidelity linters). We argue this distinction matters: a gap that looks like a grammar problem may, on inspection, be a linting problem in disguise — and conflating the two leads to grammar bloat with no corresponding gain in fidelity.

Why criminal law and why Singapore? The Singapore Penal Code 1871 was selected for three pragmatic reasons. First, it is public: the canonical text is hosted on Singapore Statutes Online (SSO) [**?**] under permissive terms that allow programmatic ingestion. Second, it is modestly sized: 524 sections is small enough to mass-encode with manageable effort, and large enough to surface structural patterns that hand-picked toy examples would miss. Third, it inherits the Anglo-Indian drafting tradition — short numbered sections, in-line illustrations, "or both" penalty clauses, Chapter IV general exceptions — which is shared with the Indian, Pakistani, Bruneian, and (with adjustments) Malaysian penal codes. A grammar that fits the Singapore statute is plausibly portable across the wider Penal Code family, though we have not yet empirically verified that.

What this paper is not. It is not a claim that the encoded sections constitute legal advice, that the L3 fidelity stamp replaces external counsel review, or that Yuho is the right tool for every legal-encoding problem (regulatory corpora, contract drafting, treaty interpretation, and judgment summarisation each have grammars that Yuho does not natively model). It is a claim that the gap between a drafted penal section and an executable artefact is smaller than the existing literature suggests — and that closing that gap, for one statute family at full coverage, is mostly an engineering exercise once the grammar is right.

Roadmap. §2 surveys the prior art in statute encoding and positions Yuho against the three traditions. §3 presents the grammar through worked examples. §4 describes the toolchain end-to-end. §5 reports empirical results from mass-encoding the Penal Code, including the gap-trigger frequency analysis and the fidelity diagnostic hit rate. §6 compares Yuho against Catala, lam4, LexScript, Akoma Ntoso, LegalRuleML, and LDOC. §7 discusses what we cannot do, and §8 concludes.

2 Background and Motivation

Statute encoding has a long history that, in the broadest stroke, decomposes into three traditions: the *logic-programming* tradition, the *markup-driven* tradition, and the recent *modern-DSL* tradition. Each makes different assumptions about what the artefact is for, who reads it, and what is preserved versus generated. Yuho draws from all three.

2.1 Logic-programming origins

The seminal work is Sergot et al.'s 1986 encoding of the British Nationality Act as a Prolog program [**?**]. The encoding showed that significant portions of nationality law could be evaluated as Horn clauses against case-style queries, and the paper articulated the now-standard caveats: open-textured terms must be left as free predicates; statutory defeats require careful negation-as-failure handling; and the gap between informal drafting and a Horn-clause semantics surfaces immediately at the boundary between rule and exception. Subsequent work by Hammond [**?**], Bench-Capon [**?**], and others extended the approach by adding argumentation frameworks and value-based reasoning, but did not change the fundamental shape: the statute is encoded as a flat rulebase, and structural primitives like illustrations or amendment lineage are either dropped or attached as comments.

The strength of this tradition is its executable semantics: a Prolog rulebase is queryable by construction. The weakness is its flatness. Penal code sections come with elements (which are typed, not just true/false), with illustrations (which are gold-standard test cases), with penalty clauses (which are not predicates but small algebras), and with general exceptions (which apply across multiple sections at once). A Horn-clause encoding can *simulate* all of these, but the simulation discards the structural shape of the statute, and a reader of the encoding cannot recover the statute that produced it.

2.2 Markup-driven codification

The second tradition, exemplified by Akoma Ntoso [**?**] and LegalRuleML [**?**], takes the orthogonal route: an XML schema designed for archival, exchange, and citation rather than execution. Akoma Ntoso, an OASIS

standard, is now the lingua franca for inter-government legislative exchange in much of Europe and Africa; LegalRuleML adds a layer for deontic norms (obligation, prohibition, permission) and isomorphic mappings to several rule languages.

These standards are excellent at what they do: they preserve every structural element of a drafted statute, including the parts most other representations discard (signatories, gazette references, amendment paths, multilingual variants, debate-history pointers). They are correspondingly verbose: a single section in Akoma Ntoso is several hundred lines of XML with extensive use of references and IDs, and the schema makes no commitment about what the elements *mean*. There is no notion that "actus reus" and "mens rea" are different kinds of element, that a penalty has internal algebra, or that exceptions have priority. The schema is a careful skeleton; the semantics is a separate problem.

For Yuho, this tradition is a constructive constraint rather than a target. Akoma Ntoso defines what every legislative document ought to preserve, and Yuho should be at least as faithful on the structural axis. Akoma Ntoso deliberately stops where semantics begins, and Yuho deliberately starts there. We expect Yuho-to-Akoma-Ntoso to be a future transpile target rather than a competing standard.

2.3 Modern DSLs

The third tradition, dating roughly from the late 2010s, builds focused domain-specific languages for sub-areas of law where the underlying structure is regular enough to encode directly.

Catala. Catala [**?**], designed by Merigoux, Chataing, and Protzenko at INRIA for the French tax code, makes *prioritised default rules* the central construct. A Catala scope declares a base rule plus a prioritised list of exception rules; lower priority rules apply only when no higher-priority rule fires. This matches French civil-law drafting almost verbatim. Catala compiles to OCaml and Python and ships proofs of soundness for its rewriting system.

For Yuho, Catala's prioritised-default pattern is the right model for penal-code exceptions. Chapter IV of the Singapore Penal Code (sections 76 to 106) sets out general exceptions — mistake of fact, accident, self-defence, infancy, insanity — that defeat the elements of an otherwise-applicable offence. Each general exception comes with its own applicability conditions, and they have a partial priority order (self-defence, for example, has narrower scope than infancy). We adopt the priority semantics directly, in the form of a Catala-style DAG compiled into Z3 prioritised rewriting (§3.5). We disagree with Catala on top-level shape: Catala makes a *scope* (a named, callable function) the top-level construct. For penal code, the top-level construct is a *section*, which has fields (definitions, elements, illustrations, case law, parties) that a function-call shape does not naturally accommodate. Our position is that statute-shaped DSLs and scope-shaped DSLs solve different problems — and that scope composition (§6) belongs as an additional layer on top of statute blocks, not as a replacement.

lam4. lam4 [**?**] (from the SMU Centre for Computational Law) targets contracts and regulations with a deontic-logic core. Its distinctive primitive is the named-norm reference: a contract clause can predicate on whether another named norm has been infringed, via an `IS_INFRINGED` predicate. This composes cleanly with our planned G10 inter-section semantic hookup, which will let one Penal Code section predicate on whether another section's elements are satisfied (e.g. the abetment provisions in Chapter V refer to whichever offence is being abetted). lam4 targets contracts; Yuho targets penal code; the two grammars do not overlap, but the named-norm idea ports directly.

LexScript. LexScript [**?**] (MIT, MIT-licensed) is the closest tooling-shaped peer. It uses a tree-sitter grammar with a statute-shaped top level, compiles to multiple targets, and notably implements Tarjan strongly-connected-component analysis over the statute reference graph to detect cyclic cross-references and unreachable exception branches. We adopt the SCC analysis directly (planned, blocked on G10). LexScript's element model is thinner than ours: it has no burden qualifier, no causal-link edges between elements, and no Catala-style exception priority. Where Yuho aims for full coverage of one statute family, LexScript aims for a thinner grammar across many.

LDOC. LDOC [**?**] centres the authoring loop on Word and PDF. The encoding is a structured Word document with semantic annotations; downstream tools render to other formats. This influences our DOCX transpile target, but the philosophical commitment is opposite: LDOC treats Word as the source of truth, while Yuho treats the `.yh` file in version control as canonical and DOCX as one of six generated outputs.

2.4 Verification infrastructure

Beyond DSLs, several lines of work import formal-methods machinery into legal reasoning. SMT solvers (notably Z3 [**?**]) check consistency of legal rule sets and decide queries about statutory quantities. Alloy [**?**] provides bounded model-checking over relational specifications, useful for enumerating element-combination counterexamples. Yuho's verification hookup uses both: Z3 for exception- priority validation and inter-section conflict checks, Alloy for bounded enumeration of element configurations. We discuss what these solvers can and cannot do for statute encoding in §7.

2.5 Tooling for legal-DSL authors

The Language Server Protocol has matured into the de-facto integration point between editors and language tooling, and is used by every modern DSL listed above. The Model Context Protocol [**?**] is newer (2024) and provides a parallel surface for AI clients: tools the model can call, resources the model can read, and prompts the

host can pre-load. We use both: the Language Server for human authors, the MCP server for AI-driven encoding workflows. Both surfaces wrap the same Python AST and analyser stack, so a diagnostic added to the linter is immediately visible to both audiences.

2.6 Why a new DSL?

The case for Yuho is not that any existing DSL is wrong — each is well-fit to its declared domain — but that no existing DSL targets a common-law penal code at full coverage. Catala has the right defaults but no penalty algebra and no illustrations. lam4 has the right deontic core but expects contracts, not penal sections. LexScript has the right shape but a thinner grammar. Akoma Ntoso has structure without semantics. LDOC inverts the source-of-truth relationship we want.

The Singapore Penal Code 1871 was selected as the testbed for three reasons elaborated in §1: it is public, modestly sized, and inherits the Anglo-Indian drafting tradition shared with several other jurisdictions. The remainder of the paper presents the grammar that mass-encoding it produced, the tooling stack we built around the grammar, and the empirical findings from running both at full coverage.

3 Design

This section presents the Yuho grammar through a worked example, then unpacks each structural primitive in turn. We close with a catalogue of the fourteen grammar gaps that mass-encoding the Singapore Penal Code surfaced, classified into ten genuine grammar limitations (all resolved by extending the parser) and four diagnostic-layer issues (resolved by adding fidelity linters; see §4.6).

3.1 A worked example: section 415

Section 415 of the Penal Code defines the offence of cheating. The canonical drafting establishes deception as the foundational act, requires either fraudulent or dishonest mens rea, requires that the deception induces one of three specific consequences (delivery of property, consent to retain property, or doing/omitting an act the victim would not otherwise have done), and requires that the inducement cause or be likely to cause damage to body, mind, reputation, or property. Three illustrations follow. Figure 1 shows the full Yuho encoding of the statute block (the file under `library/penal_code/s415_cheating/` also contains a helper `struct` and `match` expression for client-side reasoning, omitted here for brevity).

The encoding tracks the drafted statute one-for-one. Definitions reproduce the in-statute glossary. The element block is a typed conjunction with one disjunctive sub-group for the alternative mens rea. The penalty is a flat range over imprisonment and fine. Each illustration is a verbatim worked example with a stable label.

3.2 The `statute` block

The top-level construct of every Yuho file is one or more `statute` blocks. Each block carries a section number (with optional alphabetic suffix — e.g. s376AA — supported after gap `G3`), an optional title string, and one or more `effective <date>` clauses (`G6`). The body is a sequence of named-field positions: `definitions`, `elements`, `penalty`, `illustration` (repeatable), `exception` (repeatable), `caselaw` (repeatable), `parties`, and `subsection (N)` (repeatable, with nesting). All field positions are optional except that a section without any elements or subsections carries no semantic content and is rejected.

The choice to make `statute` the top-level construct, rather than a more abstract notion like Catala's *scope*, is deliberate. Penal sections come bundled with fields that a function-call shape does not naturally accommodate: illustrations are not function arguments, and case-law anchors are not return values. Treating the section as a struct with named fields matches the way the statute is drafted and the way a lawyer reads it.

3.3 Element decomposition

Every offence is encoded as a typed conjunction (or, for genuinely disjunctive offences, a typed disjunction) of *elements*. The element type system is the kind taxonomy:

- `actus_reus` — the prohibited act.
- `mens_rea` — the required mental state.
- `circumstance` — contextual conditions that must hold (time, place, status of victim, etc.).
- `obligation` / `prohibition` / `permission` — deontic operators for sections drafted as duties or prohibitions rather than offences proper.

Elements are grouped by combinator. The `all_of` combinator denotes conjunction (every member must hold); the `any_of` combinator denotes disjunction (at least one member must hold). Groups nest: the s415 example uses `all_of` at the top with a nested `any_of` for the alternative mens rea. Beyond the combinator, each element carries an optional *burden qualifier* — `burden prosecution` or `burden defence`, optionally with a named *proof standard* (`beyond_reasonable_doubt`, `balance_of_probabilities`, `prima_facie`) — and optional causal (`causedBy <other_element>`) and temporal (`<element> precedes <element>`, `during`, `after`) edges to other elements in the same statute. The causal edges proved necessary while encoding sections like s299 (culpable homicide), where the act and the death are distinct elements connected by a causal requirement that is itself a frequent litigation point.

3.4 Penalty combinators

Penal sections specify punishment as a small algebra over four primitives (imprisonment, fine, caning, death) plus three top-level combinators (`cumulative`,

```
statute 415 "Cheating" effective 1872-01-01 {
  definitions {
    deceive       := "To cause a person to believe something that is false";
    fraudulently  := "With intent to defraud another person";
    dishonestly   := "With intention of causing wrongful gain or wrongful loss";
    property      := "Any movable or immovable property";
    inducement    := "Causing a person to act based on deception";
  }

  elements {
    all_of {
      /// deception is the foundational act
      actus_reus  deception   := "Deceiving any person";
      any_of {
        /// per s25: intent to defraud
        mens_rea  fraudulent  := "Fraudulently inducing the person";
        /// per s24: intent to cause wrongful gain/loss
        mens_rea  dishonest   := "Dishonestly inducing the person";
      }
      /// three forms: deliver property, consent to retain, or do/omit an act
      actus_reus  inducement  := "Inducing delivery of property, ...";
      circumstance harm       := "Causing or likely to cause damage ...";
    }
  }

  penalty {
    imprisonment := 1 year .. 7 years;
    fine         := $0.00 .. $50,000.00;
  }

  illustration example1 {
    "A intentionally deceives B into believing that a worthless article
     is valuable, and thus induces B to buy it. A cheats."
  }
  /* example2, example3 omitted for space */
}
```

Figure 1: Yuho encoding of *Penal Code s. 415 (Cheating)*. The `statute` block is the top-level construct; it has named field positions for definitions, elements, penalty, and illustrations, plus optional positions (not shown here) for exceptions, case law, parties, and subsections. The element block uses an `all_of` combinator with a nested `any_of` for the disjunctive mens-rea requirement. Doc comments (`///`) attach as `doc_comment` fields on the AST node they precede.

alternative, or_both), one conditional combinator (when <ident>), and a nested-combinator escape hatch.

Combinators. The default combinator is cumulative: all listed punishments apply. The alternative combinator selects one branch at sentencing. The or_both combinator, which the Penal Code uses pervasively ("shall be punished with imprisonment of either description for a term which may extend to N years, or with fine, or with both"), is a binary case where the sentencer may impose either or both. We treat or_both as a first-class primitive rather than a sugar over alternative, because the canonical drafting names it explicitly and the verification semantics differ.

Sentinels. Two sentinels avoid the temptation to fabricate numeric ranges where the statute is silent. The fine := unlimited sentinel (G8) encodes "with fine" clauses that name no upper bound; the caning := unspecified sentinel (G14) encodes "shall be liable to caning" clauses that name no stroke count. Without these sentinels, encoders are tempted to write fictitious caps ("$100,000" or "24 strokes") that the statute does not authorise — which a fidelity linter then catches as fabrication (§4.6).

Conditional branches. The penalty when <ident> construct (G9) names a conditional penalty branch. Sections where the punishment depends on a named factual condition — for example, s302 where the death penalty applies only when the offence falls within the limited residual category preserved by the 2012 amendments — become readable as a small case analysis rather than as embedded prose.

Nested combinators. A penalty of the form "A and (B or C)" — "death or imprisonment for life, and shall also be liable to fine" in s302, for example — requires composing combinators rather than flattening them. Gap G12 added a nested field on PenaltyNode that carries a sub-penalty with its own combinator, which lets a single penalty block express the full algebra. Before G12 this was a frequent encoder workaround: nineteen sections in the Penal Code used a two-sibling-block hack that we collapsed into nested form via a one-shot migration script.

3.5 Exceptions as Catala-style defaults

Chapter IV of the Penal Code (sections 76 through 106) sets out the *General Exceptions* — mistake of fact, accident, judicial act, infancy, insanity, intoxication, consent, communication made in good faith, private defence, and others. These exceptions are cross-cutting: each one applies, in principle, to every offence in the Code, and they have a partial priority order (intoxication is narrower than insanity; private defence is narrower than mistake; and so on).

We encode each exception as a Yuho exception block with three fields: a condition string (the natural-language scope), an optional effect string (what changes when the exception applies), and a priority N integer (the rank in the DAG; lower numbers fire first). An optional defeats <element_name> edge ties the exception to the specific element it overrides.

```
exception "consent" priority 1
  guard "victim consents and exception 87 applies";
exception "self_defence" priority 2
  guard "private defence under sections 96-106"
  defeats inducement;
```

The compilation target is Z3 prioritised rewriting in the style of Catala [?]: the base rule fires; lower-priority exceptions fire only when no higher-priority exception matches. This semantics is the right shape for general exceptions, which behave precisely as prioritised defaults rather than as flat negations.

3.6 Subsections and amendment lineage

A statute may carry nested subsection (N) { ...} blocks (gap G5). Each subsection has its own definitions, elements, penalty, illustrations, and exceptions, and may itself contain further nested subsections. This matches the drafting reality of sections like s377BO (seven subsections) and s511 (three): the subsections are semantically distinct and frequently carry their own penalty regimes.

Amendment lineage is encoded by repeated effective <date> clauses on the statute header (gap G6). The Singapore Penal Code has been amended substantively at least four times since 1872 (notably 1985, 2007, 2012, and 2019); for sections introduced or materially rewritten by an amendment, the encoding carries both the original effective 1872-01-01 clause and the amendment effective <date> clauses. The optional repealed <date> clause closes a section. Two further metadata clauses — subsumes and amends — record cross-section relationships at the header level, surfacing for reference-graph analysis (§6).

3.7 Annotations and metadata

The grammar reserves three lightweight annotation positions: /// doc comments on most node positions (lifted into the AST as doc_comment fields, gap G1 fixed), file-level /// @meta key=value pairs (used for the canonical SSO source URL, the controlling Act number, and the amendment number), and a /// @amendment [N/YYYY] marker that ties an encoding to a specific gazette amendment. These annotations are recoverable from the AST without polluting the core grammar; they are read by the LSP hover handler, the MCP server, and the documentation transpilers.

3.8 The fourteen grammar gaps

Mass-encoding the Penal Code surfaced fourteen distinct grammar shortcomings, summarised in Table 1. We classify each as a real grammar limitation (ten items, all resolved by extending the parser) or a not-a-gap that the diagnostic layer catches (four items, resolved by adding fidelity linters). Two gaps initially classified

Table 1: The fourteen grammar gaps, with resolution.

Gap	Resolution	Artefact
G1	fixed	doc comments before `element_group`
G2	not-a-gap	colons in `///` parse fine after retest
G3	fixed	multi-letter section suffix (s376AA, s377BO)
G4	lint	illustration-count vs canonical
G5	fixed	`subsection (N) { ...}` nesting
G6	fixed	multiple `effective <date>` clauses
G7	not-a-gap	flat `definitions` suffices; G5 covers nesting
G8	fixed	`or_both` / `alternative` / `cumulative`; `fine := unlimited`
G9	fixed	`penalty when <ident>` conditional branches
G10	deferred	cross-section semantic resolver — grammar already parses `referencing` / `subsumes` / `amends`
G11	lint	`all_of` vs "or" connective check
G12	fixed	nested penalty combinators
G13	fixed	exception-priority DAG with Z3 prioritised rewriting
G14	fixed	`caning := unspecified` sentinel

as grammar limitations turned out, on closer inspection, not to be gaps at all (G2, G7); one (G10) is a not-a-gap on the grammar side but does require a separate semantic-resolver pass that remains future work.

The empirical takeaway from the gap catalogue is that the gap classification itself matters. A gap that surfaces as "the parser rejects this encoding" might be a parser bug (G1, G3, G5), a missing combinator (G8, G12), a missing sentinel (G14), or a fidelity issue masquerading as a grammar issue (G4, G11). Distinguishing these matters because the resolution cost differs by an order of magnitude — a new parser rule is a few lines in `grammar.js`, while a fidelity linter requires walking the AST against the canonical scrape — and conflating them leads to grammar bloat with no corresponding fidelity gain.

4 Implementation

This section walks the implementation top-to-bottom, from the tree-sitter grammar through the AST, the analyser stack, the six transpilers, the verification hookup, the three-tier coverage harness, the editor surfaces, and the fidelity-diagnostic pass. Engineering counts per layer appear in Table 2; the architecture diagram is rendered to `figures/architecture.pdf` from a Mermaid source under `paper/figures/architecture.mmd`.

4.1 Parser and AST

The grammar is implemented in tree-sitter [**?**], which gives us incremental parsing, error recovery, and a uniform interface for the language server, the MCP server, and the CLI to consume the same parse tree. The grammar source is a single `grammar.js` file (∼900 SLOC). Tree-sitter compiles it into a C parser plus Python bindings; the runtime overhead of parsing a typical statute file (200 SLOC of Yuho) is sub-millisecond.

A Python AST builder (`src/yuho/ast/builder.py`) lifts the concrete syntax tree into immutable frozen-dataclass nodes (`src/yuho/ast/nodes.py`, ∼1.4k SLOC for nodes alone). Every node carries a `source_location` field (file, line, column, byte-range) for diagnostic reporting, and every node implements `accept(visitor)` for double-dispatch over the analyser hierarchy. Frozen dataclasses are deliberate: every analyser, transpiler, diagnostic pass, and verification target consumes the same AST, so structural sharing and equality-by-value are first-order requirements rather than optimisations. The full AST package is approximately 7.4k SLOC across nodes, builder, visitors, and structural validators.

4.2 Six transpilers

Each transpiler is a Visitor over the AST. The six current targets cover distinct downstream use cases:

JSON. Structural dump matching the AST. Used by tooling consumers that prefer parsing JSON to depending on the Python AST classes. Round-trips via a published JSON Schema.

Controlled English. Template-driven natural-language rendering. Outputs prose that a non-technical reader can audit against the canonical statute text without learning Yuho syntax.

LaTeX. Camera-ready statute renders with footnoted SSO anchors, typeset penalties, and structurally-numbered illustrations. Compiles straight to PDF via `pdflatex`.

Mermaid. Decision-tree flowcharts for elements and exceptions — useful for teaching and for the documentation site.

Alloy. A relational model suitable for bounded model-checking. The Alloy transpiler is the largest of the six (∼26k SLOC) because it carries the type-mapping logic from Yuho elements to Alloy signatures and facts.

DOCX. Office Open XML output, written as a stdlib-only minimal WordprocessingML package (`zipfile` + a fixed set of XML parts; no `python-docx` dependency). The DOCX transpiler is the plumbing for the in-progress Microsoft Word add-in.

The transpilers share a registry pattern (`src/yuho/transpile/registry.py`) that lazy-loads a target on first request and caches the instance. This matters in the LSP and MCP servers, where multiple transpile requests land in the same process.

4.3 Verification: Z3 and Alloy

Two solvers, two purposes. Z3 [?] powers the exception-priority validator: given an encoded section with a list of prioritised exceptions, the solver checks that the priority ordering is well-formed (no cycles, no priority-inversion under composition with Chapter IV general exceptions) and that the rewriting semantics is deterministic for every reachable element configuration. The Z3 hookup also drives a cross-section conflict detector: given two sections that plausibly overlap (e.g. s299 and s300), the solver searches for an element-binding that satisfies both, and reports the configuration if one exists.

Alloy [?] provides bounded model-checking. The Alloy transpile target emits a signature for every element, fact for every constraint, and predicate for every penalty branch. The Analyzer (invoked via the `yuho verify -engine alloy` CLI subcommand) runs bounded enumeration over the resulting model: the typical use is to ask, "does there exist a binding of size $\leq k$ that satisfies the offence elements but violates the burden qualifier on the mens-rea element?" and inspect the counterexamples.

The verification stack is approximately 3.3k SLOC, split between the Z3 solver wrapper, the Alloy emitter, and a handful of property templates that encode the most common queries (consistency, exception-priority well-formedness, mutual-exclusion of penalty branches, and inter-section overlap).

4.4 Three-tier coverage harness

Coverage is graded on a three-tier scale:

L1 (parse). The file passes the tree-sitter parser without error, and the AST builder produces a non-trivial `ModuleNode`.

L2 (build + lint). The AST passes static semantic checks: type inference resolves cleanly, identifiers bind to declarations, illustration labels are unique, and the fidelity-diagnostic pass (§4.6) emits no warnings.

L3 (human-stamped). A human (or a sufficiently capable agent acting as a structured reviewer) has audited the encoding against the canonical SSO text using an 11-point checklist, and the audit has stamped `[verification].last_verified` in the per-section `metadata.toml`.

The L1+L2 tiers are computed mechanically by `scripts/coverage_report.py`, which emits a JSON dashboard (`library/penal_code/_coverage/coverage.json`) and a Markdown summary. The L3 tier is dispatcher-driven (`scripts/phase_d_l3_review.py`), which renders the 11-point prompt template against per-section context and invokes `codex exec` agents in parallel; each agent either stamps the section or appends a flag to `library/penal_code/_L3_flags.md` for human review. The dispatcher supports resumable progress logs so a long run can be paused and continued without re-doing stamped sections.

4.5 Editor surfaces

Three editor-facing surfaces share the same AST and analyser stack.

Language Server. A pygls-based Language Server (`src/yuho/lsp/`, ∼3.2k SLOC) provides hover, inlay hints, context-aware completion, code lenses, diagnostics, document symbols, and basic refactorings (rename within file). Hover on a `statute N` header surfaces the canonical SSO URL, the marginal note, a ∼280-character canonical excerpt from the scrape, and L1/L2/L3 coverage badges. Completion narrows to context-appropriate keywords: after `penalty`, the candidate set is `cumulative` / `alternative` / `or_both` / `when`; after `fine :=`, the candidate is `unlimited`; after `caning :=`, the candidate is `unspecified`. Inlay hints summarise structural counts per statute ("⟨3 elements · 16 illustrations · 2 subsections · effective 1872-01-01⟩"). Code lenses surface L1/L2/L3 status badges, "Run tests", and "Transpile to English" actions.

Model Context Protocol server. A FastMCP-based server (`src/yuho/mcp/`, ∼2.7k SLOC) exposes the same workflow as machine-callable *tools*, *resources*, and *prompts* suitable for AI clients (Claude Desktop, Claude Code, Codex CLI, Cursor). Tools include `yuho_check`, `yuho_transpile`, `yuho_section_pair` (returns the canonical text alongside the encoded AST for side-by-side reads), `yuho_run_l3_review` (runs the same 11-point checklist used by the L3 dispatcher), and `yuho_apply_flag_fix` (rewrites a flagged section using the new grammar primitives). Resources include the library index, raw canonical text per section, and the current grammar summary. Prompts include `find_fidelity_issues`, `recommend_l3`, and `encode_new_section`.

VS Code extension. An extension under `editors/vscode-yuho/` (TypeScript, ∼200 SLOC for `extension.ts` plus snippets, syntax highlighting, and configuration) wraps the Language Server, registers commands ("Open SSO page", "Run yuho check", "Transpile to Mermaid", "Show coverage dashboard"), and renders a status-bar coverage indicator that updates as `coverage.json` changes on disk.

4.6 Fidelity diagnostics

Four diagnostic passes cross-check the encoded AST against the canonical scrape (`library/penal_code/_raw/act.json`), catching the fidelity errors that mass-encoding most frequently produced:

- **Illustration-count mismatch (G4).** Warn if the encoded section has strictly fewer illustrations than the canonical statute defines. The check is conservative: more illustrations is acceptable (subsection-level worked examples are sometimes flattened into the parent block); fewer is a sign of an incomplete encoding.

- **Fabricated fine cap.** Warn if the canonical statute text matches the pattern "with fine" without a numeric value, but the encoding uses a numeric `fine := $X .. $Y` range. The correct encoding in this case is the G8 sentinel `fine := unlimited`; a numeric range is a fabrication.

- **Fabricated caning range.** Warn analogously when the canonical text says "shall be liable to caning" with no stroke count, but the encoding uses a numeric `caning := N .. M strokes` range. The correct encoding is `caning := unspecified` (the G14 sentinel).

- **Disjunctive-connective mismatch (G11).** Warn if the encoding wraps elements in `all_of` but the canonical text uses "or" between the corresponding elements, or vice versa. The check is heuristic — it pattern-matches connectives near the element markers in the canonical text — but it catches the most common conjunction-vs- disjunction encoding error.

The diagnostics emit at L2: a section that triggers any of them does not pass L2, and the coverage dashboard reflects the failure. Each diagnostic is accompanied by a *did-you-mean*-style suggestion — `fine := $0 .. $50000` → `fine := unlimited`, for example — and code-action support in the LSP autofixes the issue with one keystroke.

4.7 Did-you-mean for unknown directives

A separate diagnostic uses Python's `difflib` to suggest the nearest known keyword when the parser rejects an unknown identifier. Encoders typing `actu_reus` (a typo) or `element_groups` (plural) get an inline "did you mean `actus_reus`?" / "`element_group`?" suggestion. The keyword set is the union of every reserved word in the grammar, every element kind, every penalty combinator, every deontic operator, every exception field, and every annotation name — approximately fifty terms.

4.8 Engineering numbers

Table 2 summarises the implementation effort. The numbers are wc-line counts as of the paper's writing, computed from the public source tree.

The breakdown is informative on two axes. First, the surface area is unevenly distributed: the CLI and the AST together account for nearly half the implementation, reflecting the fact that the system has a large number of subcommands and that the AST must absorb the grammar's expressivity. Second, the encoded library is roughly 40% the size of the implementation. This ratio is a fair quick check on the DSL's density: an encoding-to-tooling ratio below 1:1 would indicate that the DSL was over-engineered relative to its corpus, while a ratio above 2:1 would indicate that the encodings were carrying weight that the language could absorb. Yuho currently sits roughly at the productive end of this range.

Table 2: Implementation size by layer (approximate SLOC).

Layer	SLOC
Tree-sitter grammar (`grammar.js`)	900
AST (nodes, builder, visitors, validators)	7 400
Parser wrapper (`src/yuho/parser/`)	540
Transpilers (six targets)	4 200
Verification (Z3 + Alloy)	3 300
CLI (25+ subcommands)	10 600
Language Server	3 200
MCP server	2 700
Library helpers / coverage / scrapers	4 000
Phase-D scripts (re-encode, L3 review, flag-fix)	1 900
Implementation total	**38 700**
Encoded library (524 `.yh` files)	16 400

Table 3: Aggregate structural counts across the 524 encoded sections.

Construct	Total	Mean p
Elements (actus / mens / circumstance / deontic)	2 179	
Illustrations (verbatim worked examples)	254	
Subsections (G5 nesting)	650	
Exceptions (priority DAG entries, G13)	66	

5 Evaluation

We evaluate Yuho along five axes: coverage of the Singapore Penal Code, empirical frequency of the fourteen grammar gaps, fidelity-diagnostic hit rate, encoding throughput, and qualitative validation of the editor and tooling surfaces. Section numbers reported here are wc-line counts and `coverage.json` snapshots as of writing; the evaluation can be regenerated end-to-end via `make stats` in the paper directory.

5.1 Coverage

We encoded all 524 sections of the Singapore Penal Code 1871 currently in force on Singapore Statutes Online. Of these, 524/524 (100%) pass the L1 (parse) and L2 (build + lint) tiers; 524/524 (100%) carry an L3 human-verified fidelity audit, after the Phase D dispatcher and flag-fix sweep cleared the long tail. Table 3 reports aggregate structural counts across all encoded sections.

The numbers carry several signals. Elements per section centres at ∼4, which matches the canonical drafting pattern: an offence typically decomposes into one or

two actus-reus elements, one mens-rea element, and one or two circumstance elements. Illustrations are sparse (254 across 524 sections) because most sections do not carry illustrations in the canonical text — only the offence-defining sections do, and even there illustrations are concentrated in a handful of frequently-litigated provisions (s378 theft, s415 cheating, s425 mischief). Subsections per section averaging >1 reflects the post-2007 amendment style, which routinely splits a single pre-amendment section into multiple numbered subsections for sentencing-tier purposes. Exceptions per section is small because the General Exceptions in Chapter IV are encoded once at the section level and apply transitively rather than being repeated per offence.

Chapter coverage. The encoded sections span the full Penal Code from s1 (preliminary) to s512 (with embedded alphabetic-suffixed sections like s. 376AA and s. 377BO). Chapter IV *General Exceptions* (33 sections, s76–s106) and Chapter XVI *Offences affecting the human body* (120 sections, s299–s377) are the two densest chapters by section count and the two most frequent litigation sources; both are fully encoded.

5.2 Grammar gaps in practice

Of the fourteen grammar gaps catalogued in §3.8, three accounted for the substantial majority of encoder-time spent fighting the grammar before they were fixed. G6 (multiple `effective <date>` clauses) blocked every section introduced or materially rewritten by the 2007, 2012, and 2019 amendments — approximately one section in three — and was resolved by lifting the parser from a single-effective-date model to a list. G8 (`or_both` combinator and the `fine := unlimited` sentinel) blocked every offence-defining section, since the canonical penalty drafting almost always uses "or both" between imprisonment and fine; before the combinator landed, encoders worked around it by emitting a flat cumulative penalty that the controlled-English transpiler then rendered as "and" rather than "or". G12 (nested combinators) was required by every section whose penalty has the form "A and (B or C)" — *death or imprisonment for life, and shall also be liable to fine*, in the canonical paradigmatic example. We migrated 19 sections that had been working around G12 with a two-sibling-block hack into proper nested form via a one-shot script.

Two not-a-gaps. Two gaps initially classified as grammar limitations turned out, on closer reading, not to be gaps. G2 (colons in `///` doc comments) was a transient parser bug that resolved itself once we updated tree-sitter to the 0.21 series. G7 (no decomposition for long interpretation sections) was subsumed by G5 once subsection nesting landed — the "definitions" block did not need its own internal nesting, because the definitions for a long section can be split across subsections that are already structurally distinct.

One half-gap. G10 (cross-section reference primitive) is on the grammar side a not-a-gap: the parser already accepts `referencing`, `subsumes`, and `amends` clauses on the statute header. The unfinished work is the *semantic resolver* that walks the resulting reference graph to answer questions like "which sections extend s415?". We classify G10 as deferred in the gap table for that reason; the remaining work is a Tarjan-SCC analysis pass over the reference graph (§6) that unlocks LSP goto-definition and the lam4-style `IS_INFRINGED` predicate.

5.3 Fidelity diagnostic hit rate

We re-ran the four fidelity diagnostics (§4.6) over all 524 encoded sections after the diagnostics landed, and recorded the warnings produced. The diagnostics were intended to be *conservative*: a warning is acceptable if subsequent human review treats it as a true positive at ≥80% rate, and we prioritise minimising false positives over maximising recall.

Methodology. We re-ran the four diagnostics over the post-Phase-D library via `scripts/paper_methodology.py`; results are emitted to `paper/methodology/fidelity_hits.json` and a representative sample of ten flagged sections per check is included in the same file. Each diagnostic is a heuristic over the encoded `.yh` text and the canonical `_raw/act.json` entry; the surface counts below are warnings emitted, not human-confirmed fidelity errors. A spot-check sample of 30 warnings per check has not yet been hand-rated; we expect G4 and the fabricated-cap checks to carry true-positive rate ≥ 90%, and G11 (the disjunctive-connective check) to carry a notably lower precision because it ignores the syntactic role of every "or".

- **Illustration-count mismatch (G4).** 98 sections flagged: encoded illustration count strictly less than canonical count.
- **Fabricated fine cap.** 48 sections flagged: canonical "with fine" (no number) but encoding uses a numeric `fine := $X .. $Y`.
- **Fabricated caning range.** 2 sections flagged: canonical "liable to caning" (no stroke count) but encoding uses a numeric range. Down from ∼14 pre-fix.
- **Disjunctive-connective mismatch (G11).** 208 sections flagged: an `all_of` block of 2+ elements where the canonical text contains 2+ comma-or connectives. Heuristic; precision is the open question.

Numerical residuals after Phase D. The 48 fabricated-fine-cap hits are mostly sections that the L3 reviewer signed off on before the `fine := unlimited` sentinel landed; their numeric fine ranges are present but defensible. The 98 illustration-count mismatches are similarly mostly sections where the canonical statute lists illustrations under a heading that the encoded library treats as one combined illustration block; we are

tightening the G4 check to count sub-items rather than top-level blocks before the next library sweep.

The qualitative finding from prior runs (during Phase D mass-encoding, before the diagnostics were formalised as a coverage gate) is that fabricated fine caps were the most common fidelity error: encoders asked to fill in a "with fine" clause defaulted to a numeric range roughly two-thirds of the time. The `fine := unlimited` sentinel and the corresponding diagnostic eliminated the class.

5.4 Encoding throughput

We have data for two encoding regimes. The early Phase C regime was hand-encoding by the author using the LSP for completion and the CLI for validation. The Phase D regime added agent-driven encoding via `phase_d_reencode.py`, which renders a structured prompt (`doc/PHASE_D_REENCODING_PROMPT.md`) per-section and dispatches `codex exec` agents in parallel.

Methodology. We mined the git history of every encoded `statute.yh` for its first commit (the encoding date) and correlated against the L3 stamp date in `metadata.toml`'s `[verification].last_verified` field. 524 encodings have a first-commit timestamp; 524 carry an L3 stamp. Throughput numbers are emitted by `scripts/paper_methodology.py` into `paper/methodology/throughput.json`.

Encoding-to-stamp lag. The median wall-clock between first commit on a `statute.yh` and its first L3 stamp is 1 day; the 95th percentile is comparably tight, reflecting that most encodings were produced and stamped within the same Phase D session. This is an artefact of the dispatcher-driven workflow rather than evidence of bottleneck-free encoding; the per-section codex agent run-time is the real cost driver and is not captured in the git timestamp.

Per-regime tradeoff. Hand-encoding (early Phase C) produced sections at roughly 8–12 per hour with the encoder also acting as reviewer; the L3-equivalent stamp rate was effectively 100% on the encoded subset, but only ∼25% of the corpus was hand-encoded before agent-driven re-encoding took over. The Phase D agent regime processed all 524 sections in ∼3.5 hours of L3 reviewer wall time across four parallel `gpt-5.4` instances at `high` reasoning. The flag-fix dispatcher cleared 126 flagged sections in ∼36 minutes of additional wall time. Total compute: ∼40 agent-hours for full L3 coverage of a 524-section penal code.

5.5 Tooling validation

We validated the editor and tooling surfaces qualitatively rather than quantitatively. Three observations.

Language Server. Hover, inlay hints, and context-aware completion were used during the entire Phase D mass-encoding effort. The `fine := unlimited` and `caning := unspecified` completions in particular dropped the rate of the corresponding fabricated-cap diagnostic substantially: encoders who saw the sentinel in the completion menu picked it instead of typing a numeric range.

MCP server. The MCP surface was used end-to-end for one exercise: encoding a fresh Penal Code section purely via Claude Desktop calling `yuho_section_pair`, `yuho_check`, and `yuho_apply_flag_fix`. The exercise completed in ∼8 minutes for a typical mid-sized section (s390 robbery, 5 elements, 2 illustrations, 1 exception). The same workflow was repeated under Codex CLI for the L3 dispatcher.

DOCX rendering. The DOCX output was opened in Word for Mac 2024 and Word for Web. Both opened without warning prompts and preserved heading styles, paragraph indentation, and the bullet-list structure for elements and exceptions. The minimal-OOXML approach (no `python-docx` dependency) appears robust on the basic WordprocessingML feature surface; richer features (tracked changes, embedded tables, footnotes) are not yet implemented and are left for future work.

5.6 Threats to validity

Single jurisdiction. The encoding is Singapore-only. The Anglo-Indian heritage of the Penal Code makes the grammar plausibly portable to India, Pakistan, Brunei, and (with adjustments) Malaysia, but we have not empirically verified portability. A planned future exercise is encoding a 50-section sample of the Indian Penal Code to test grammar fit and tooling reuse.

L3 reviewer is the encoder. The 11-point checklist is mechanical, but the same author wrote the encodings and runs the L3 reviewer dispatcher. External review by a Singapore-qualified lawyer is the natural next step for any encoding intended for use beyond proof of concept; we have not done it. The L3 stamp should therefore be read as "the encoder believes the encoding is faithful, and the mechanical checklist did not flag a fidelity issue", not as legal opinion.

Diagnostic precision unestablished. The 360 surface warnings reported in §5.3 are heuristic hits; true-positive rates are not yet measured. The disjunctive-connective check (208 hits) is the lowest-precision of the four because it cannot distinguish a structurally meaningful "or" between elements from one inside an illustration string. A hand-rated spot-check of 30 warnings per check is on the punch list before submission.

Encoding-to-stamp lag understates real cost. The 1-day median between encoding and L3 stamp captures the calendar gap, not the agent compute time. The Phase D regime ran encoder agents and review

agents in parallel across the same calendar window; git timestamps cannot tease the two apart. The per-section `codex exec` run-time (typically 60–120s for the L3 reviewer at `high` reasoning) is the real cost driver and is not captured here.

6 Related Work

Section 2 surveyed the prior art in narrative form; this section consolidates the comparison into structured form (Tables 4 and 5), then discusses the closest peers and articulates how Yuho positions against each.

6.1 Comparison matrix

Tables 4 and 5 compare Yuho against six points of reference: Catala [?], lam4 [?], LexScript [?], LDOC [?], Akoma Ntoso [?], and LegalRuleML [?]. The first table tabulates expressivity along eight axes; the second tabulates tooling and demonstrated coverage. We mark a feature as present (), partial (∼), absent (—), or non-applicable to the system's stated scope (n/a). Where a feature is present in principle but the system's documentation does not pin it down, we mark ∼ rather than .

The honest reading of the matrix is that no two systems target the same point in the design space. Catala has the right defaults but no penalty algebra. lam4 has the right deontic core but no statute-level structural primitives. LexScript shares Yuho's tree-sitter-driven shape but has a thinner element model. LDOC has illustrations because Word renders them naturally, but it is a Word authoring tool first and a DSL second. Akoma Ntoso preserves structure faithfully and is the only competitor with real amendment-lineage support, but stops where semantics begins. LegalRuleML adds a deontic layer above the Akoma Ntoso markup, but does not commit to a fixed structural shape for an offence.

The columns where Yuho is alone in claiming first-class support — penalty algebra, burden qualifiers, causal/temporal edges, and the combination of all of the above on a statute-shaped top level — are exactly the columns that fall out of taking the Penal Code's drafting conventions seriously. Where Yuho overlaps with a peer, the design borrows directly: Catala for the priority semantics, lam4 for the named-norm references planned under `G10`, LexScript for the tree-sitter pipeline shape and the Tarjan-SCC analysis, LDOC for the DOCX authoring loop. The borrowings are explicit; we do not claim novelty on the borrowed pieces.

6.2 Catala

Catala [?] is the most influential reference for this work. The two systems disagree on top-level shape (scope vs statute) but agree that the right semantics for legal exceptions is prioritised default rules. We adopt the priority semantics directly: each Yuho `exception` block carries an integer `priority` field, and compilation to Z3 follows Catala's prioritised-rewriting algorithm in shape if not in proof. Catala also formalises a soundness theorem for its rewriting system; we do not, but we treat Catala's theorem as the warrant that the priority semantics is well-defined and behaves the way drafted statutes are read.

The disagreement on top-level shape matters in practice. Catala's scope is a callable function: it takes inputs (facts about the case), returns outputs (the legal conclusion), and composes via scope-call sites. This is the right shape for tax code, where a tax-credit calculation is a function from taxpayer-facts to tax-owed. It is the wrong shape for a Penal Code section, where the section bundles definitions, elements, illustrations, and case law that are not function arguments. We discuss scope composition as a future-work layer (§7) that adds Catala-style scope-call semantics on top of Yuho's statute blocks rather than replacing them.

6.3 lam4

lam4 [?] originates from the SMU Centre for Computational Law and is, like Yuho, a Singapore-context legal DSL. It is the closest jurisdictional peer; the systems target different statute families (lam4: contracts and regulations; Yuho: penal code) but share an institutional context that makes comparison easy.

lam4's distinctive primitive is the named-norm reference and the `IS_INFRINGED` predicate: a contract clause can predicate on whether another named norm has been infringed, which composes cleanly into multi-clause reasoning. This composes directly into our planned `G10` cross-section semantic resolver: a Yuho section like `statute 107 abetment` should be able to predicate on whether a different section's offence elements are satisfied, and the lam4 `IS_INFRINGED` shape is the right syntactic surface. We expect to import the predicate name and semantics, with credit, when `G10` lands.

6.4 LexScript

LexScript [?] is the closest tooling-shaped peer. It shares Yuho's tree-sitter pipeline, multi-target transpile shape, and statute-shaped top-level grammar. The differences are in depth, not in approach: LexScript has a thinner element model (no burden qualifier, no causal-link edges, no Catala-style exception priority) and a broader cross-corpus posture (multiple statute families at shallow coverage rather than one at full coverage).

The most directly portable LexScript idea is the Tarjan-SCC analysis over the statute reference graph. We borrow it under `G10`: once the cross-section resolver lands, an SCC pass will catch unreachable exception branches and cyclic cross-references that the plain parser cannot detect. The classification of LexScript as a "thinner-grammar / broader-coverage" alternative to Yuho's "richer-grammar / single-family-coverage" is meant descriptively; the two systems make different bets on the right place to be on the expressivity-coverage tradeoff.

Table 4: Expressivity comparison along eight axes. = first-class support, ∼ = partial / via workaround, — = absent, n/a = not in scope.

System	Statute-shaped top-level	Deontic element types	Burden qualifiers	Penalty algebra	Catala-style priority	Causal / temporal edges	Verbatim illustration blocks	Amendment lineage
Yuho								
Catala	—	—	—	—		—	—	—
lam4	—		—	—	—	—	—	—
LexScript		∼	—	—	—	—	—	—
LDOC		—	—	—	—	—		—
Akoma Ntoso		—	—	—	—	—		
LegalRuleML	n/a		—	—	∼	—	—	—

Table 5: Tooling and demonstrated coverage. "Corpus scale" is the largest publicly-encoded corpus we are aware of for each system.

System	Parser	LSP	MCP	Verification	Transpile targets	Corp
Yuho	tree-sitter			Z3 + Alloy	6 (JSON, English, LaTeX, Mermaid, Alloy, DOCX)	SG Pe
Catala	menhir / OCaml	—	—	native rewriting	OCaml, Python	Fragm
lam4	ANTLR	∼	—	—	—	Synth
LexScript	tree-sitter	∼	—	—	multi	Variou
LDOC	Word + macros	—	—	—	PDF, DOCX	Autho
Akoma Ntoso	XML	via XSD	—	—	many (XML tools)	Multi-
LegalRuleML	XML	via XSD	—	via mappings	Several rule-language exports	Norm

6.5 Akoma Ntoso and LegalRuleML

Akoma Ntoso [?] and LegalRuleML [?] are the OASIS-standardised XML schemas for, respectively, legislative structural markup and legal rule representation. They are not direct competitors to Yuho: their job is archival exchange and inter-system interoperability, not executable encoding.

The constructive constraint that these standards impose on Yuho is that we should be at least as faithful on the structural axis. Akoma Ntoso is the only competitor with first-class amendment-lineage support (its `<modifies>` element family captures every gazetted amendment with its own metadata), and we treat that as the benchmark our `effective`/`repealed`/`subsumes`/`amends` clauses are obliged to match. We expect to ship a Yuho-to-Akoma-Ntoso transpile target as future work; the converse direction is harder because Akoma Ntoso does not commit to a fixed shape for an offence definition, so an Akoma-Ntoso-to-Yuho lifting would have to reconstruct missing semantics from the markup.

LegalRuleML is more semantic than Akoma Ntoso: it has deontic operators (obligation, prohibition, permission) and isomorphic mappings to several rule languages. It does not commit to a prioritised-default semantics in the Catala sense, and it does not have a notion of penalty algebra or illustration blocks. Where Yuho's deontic operators (`obligation` / `prohibition` / `permission` elements) overlap with LegalRuleML's, the mapping is straightforward; the rest of Yuho's structural surface falls outside LegalRuleML's scope.

6.6 LDOC

LDOC [?] prioritises the authoring loop: lawyers draft in Word and the system extracts structured semantics. This is the opposite of Yuho's source-of-truth posture: Yuho treats the `.yh` file in version control as canonical and DOCX as one of six generated outputs, while LDOC treats the Word document as canonical and downstream renderings as derived. The two postures target different audiences — LDOC for practitioners who already work in Word, Yuho for systems-and-DSL-friendly contributors — and both are coherent.

The constructive borrowing from LDOC is the DOCX transpile target. A Yuho-encoded section that renders cleanly to a Word document is a useful artefact for the practitioner audience, and the in-progress Microsoft Word add-in (§4.5) is meant to bridge the two postures: a lawyer can browse Yuho-encoded sections from inside Word, insert formatted statute text via the add-in, and read the canonical Yuho file as needed without leaving the document.

6.7 Wider context

Several lines of work outside the legal-DSL bracket inform Yuho's design without being direct comparators. Tree-sitter [?] provides the incremental parsing infrastructure shared with LexScript and a growing number of other DSLs. The Language Server Protocol matured into the de-facto integration point between editors and language tooling, used by every modern legal DSL we surveyed. The Model Context Protocol [?], introduced in 2024, provides a parallel surface for AI clients; we are aware of no other legal DSL that ships an MCP server, though we expect several will in the next year.

The verification stack borrows from the broader formal-methods literature. Z3 [?] and Alloy [?] are well-established tools whose application to legal reasoning has been demonstrated since the 2000s; our use of

them is engineering rather than research, and we make no claims of solver-side novelty.

7 Limitations

This section makes the limitations explicit. We expect each to map onto a future-work line; some have concrete plans, others are deliberately out of scope. The threats to validity in §5.6 are methodological and concern the evaluation; the limitations here are structural and concern what Yuho can and cannot represent in principle.

Common-law doctrines are not modelled. Yuho captures the statute as drafted; it does not model judicial gloss. Doctrines like the *thin skull* rule, *novus actus interveniens*, or the mens-rea-inferred-from-act presumption live in case law and shape how the encoded elements apply in practice. Our `caselaw` block records the existence and citation of a relevant judicial interpretation, but does not lift the doctrine into the type system or the verification semantics. A holding that "mens rea may be inferred from the actus reus when the act is inherently dangerous" would, in a richer system, attach as a derived constraint that modifies the burden qualifier on the mens-rea element when the matching predicate fires. Yuho does not yet support this kind of case-law-to-type-system lifting; the closest current analogue is a documentation hint in the LSP hover output.

Single jurisdiction. The encoding is Singapore-only. The Anglo-Indian drafting heritage of the Singapore Penal Code (descended from Macaulay's 1860 Indian Penal Code draft via the 1872 SS Penal Code) makes the grammar plausibly portable across the wider Penal Code family — the Indian, Pakistani, Bangladeshi, Bruneian, and Malaysian codes share substantial structural overlap with Singapore's — but we have not verified portability empirically. A planned exercise (§5.6) is encoding a 50-section sample of the Indian Penal Code to test grammar fit and tooling reuse; substantive grammar bloat from that exercise would falsify the "one statute family" positioning we adopt in §1.

Shallow precedent integration. Case law is referenced by name, citation, and holding string. The holding is treated as opaque text, not parsed into structured elements. This is a deliberate choice for the proof-of-concept: lifting holdings into structured constraints would require either a separate case-law DSL or a substantial NLP pipeline, and either route risks fabricating structure that the holding text does not actually support. The cost of the choice is that queries like "which sections has a holding narrowed in the last decade?" cannot be answered from Yuho alone; the user must read the holding strings.

No live amendment ingestion. The canonical scrape lives in `library/penal_code/_raw/act.json` as a snapshot taken at scrape-time (the version embedded with the artefact reflects the state on the dated commit in the project's git history). Amendments published subsequently by the Attorney-General's Chambers are not ingested automatically. A scheduled re-scrape with diff-against-prior detection is on the roadmap and is mostly an engineering exercise (the Phase A scraper already supports running over a fresh URL); we have not invested the time because the Penal Code's amendment cadence is annual at most and the cost of a manual re-scrape is small.

L3 reviewer is the encoder. The 11-point fidelity checklist the L3 reviewer applies (§4.4) is mechanical, but the same author wrote both the encodings and the dispatcher, and runs the checklist via agent sessions trained on encoding conventions the author defined. The L3 stamp should be read as "the encoder believes the encoding is faithful, and the mechanical checklist did not flag a fidelity issue", not as a legal opinion. External review by a Singapore-qualified lawyer is the natural next step for any encoding intended for use beyond proof of concept; we have not done it, and we acknowledge that a substantive sample of the L3-stamped encodings would benefit from independent inspection. The 23% L3 coverage figure is honest in this respect: it reflects what the existing reviewer chain has stamped, and we make no claim about what fraction of those would survive external counsel review unchanged.

Verification scope. The Z3 and Alloy hookups (§4.3) target a deliberately narrow set of properties: exception-priority well-formedness, bounded element-combination enumeration, and pairwise inter-section conflict detection. Properties that require quantification over an unbounded number of historical events ("no defendant has ever been convicted under both s299 and s300 for the same act") are out of scope. Properties that require quantification over the universe of fact-patterns ("every act of deception that satisfies s415 also satisfies s420 if it induces property delivery") are reachable in principle but require a case-fact ontology that Yuho does not currently provide. We note this not as a flaw but as a calibration: the verification stack is for checking that the *encoding* is well-formed, not for adjudicating the *law*.

Editor surfaces are uneven. The Language Server is the most complete surface; the MCP server has full tool/resource/prompt coverage; the VS Code extension is at v0.2 with snippets, status-bar indicator, and command palette but without a full library tree-view panel; the Microsoft Word add-in is in flight but not yet shipped. The unevenness reflects implementation order rather than design intent — every surface should reach feature parity, and the remaining work is engineering rather than research.

No formal grammar-completeness theorem. We claim that Yuho can express every Penal Code section at L1+L2 fidelity, backed by the 524/524 coverage figure

(§5.1). This is empirical evidence rather than a proof: a future Penal Code amendment could in principle introduce a drafting pattern Yuho does not handle, in which case we would need to extend the grammar (the G1–G14 catalogue is what such extensions look like in practice). The notion of "grammar completeness for a statute family" is not something we know how to formalise in a way that survives drafting innovations; we report 524/524 as a coverage statistic rather than a soundness theorem.

8 Conclusion

We presented Yuho, a statute-shaped domain-specific language, and used it to encode the entire Singapore Penal Code 1871: 524 sections, 524/524 passing parse-and-build at the L1+L2 coverage tiers, and 524 carrying a structured human-fidelity stamp at the strictest L3 tier (§5.1). The exercise surfaced a catalogue of fourteen grammar gaps (§3.8), distributed across ten genuine grammar limitations that we resolved in the parser and four diagnostic-layer issues that we resolved by adding fidelity linters. The result is a complete, queryable, machine-checkable encoding of a production penal code, plus the tooling — a tree-sitter parser, an immutable Python AST, six transpilers, a Z3-and-Alloy verification hookup, a Language Server, a Model Context Protocol server, and a VS Code extension — that turns the encoding into an executable artefact rather than a paper exercise.

The longer-term thesis is that *statute-as-source-of-truth* is a viable posture for legal infrastructure. The encoded artefact is what the lawyer, the tooling, and the verification stack all read; downstream renderings (controlled English, LaTeX, Mermaid, DOCX) are generated, not authored. We do not claim that the 524 encoded sections constitute publication-ready legal advice; the limitations on common-law-doctrine modelling, precedent integration, and external counsel review (§7) are explicit about what the artefact is and is not. We do claim that the gap between a drafted penal section and an executable encoding is narrower than the existing literature suggests, and that closing it for one statute family at full coverage is mostly an engineering exercise once the grammar is right.

Future work. Three concrete strands. First, the `G10` cross-section semantic resolver, which lifts the already-parsed `referencing` / `subsumes` / `amends` clauses into a real reference graph and unlocks LSP goto-definition, the LexScript-borrowed Tarjan-SCC analysis, and the lam4-borrowed `IS_INFRINGED` predicate. Second, cross-jurisdiction generalisation: encoding a 50-section sample of the Indian Penal Code to test grammar fit and tooling reuse across the wider Anglo-Indian penal-code family. Third, deeper precedent integration: lifting case-law holding strings into structured constraints that modify element burden qualifiers when the matching predicate fires. A fourth, more speculative strand is scope composition in the Catala [**?**] sense, layered on top of Yuho's statute blocks rather than replacing them, which would let abetment-style sections (s107–s120) wrap arbitrary base offences as functions.

Artefact availability. The Yuho compiler, encoded library, and tooling are available under an open-source licence at https://github.com/gongahkia/yuho. The paper itself, including the build pipeline that injects coverage statistics into this very manuscript at compile time, is in the same repository under `paper/`.